

Proyecto Final – Plataformas II

Materia

Plataformas II

Integrantes

Juan Manuel Velosa

Lina Maria Andrade

Profesor

Christian Flor

Universidad

Universidad Icesi

Fecha

Diciembre 2025

Resumen

El proyecto consiste en diseñar, desplegar y asegurar una arquitectura completa de microservicios sobre Kubernetes. Para ello, se crea un clúster funcional que permita implementar todos los microservicios del repositorio base, asegurando que las dependencias entre ellos estén correctamente configuradas.

Se organizan los entornos de desarrollo, pruebas y producción mediante namespaces, y se habilitan los servicios de Kubernetes necesarios, incluyendo un Ingress Controller con soporte TLS/HTTPS. Además, se aplican políticas de red, Service Accounts con permisos mínimos y se realizan escaneos de vulnerabilidades para garantizar la seguridad del cluster.

En cuanto a la gestión de la configuración, todos los parámetros de Spring Boot se migran a ConfigMaps, mientras que las credenciales y datos sensibles se manejan mediante Secrets, aplicando prácticas de rotación y centralización.

Introducción

El desarrollo de arquitecturas modernas basadas en microservicios requiere entornos escalables, seguros y observables, capaces de integrarse con prácticas avanzadas de despliegue y automatización. En este proyecto final del curso Plataformas II, se implementó una plataforma completa de e-commerce basada en microservicios sobre Kubernetes, abarcando desde el diseño arquitectónico hasta la automatización de despliegues y la integración continua.

El trabajo se centró en la creación de un clúster Kubernetes funcional en entornos locales, donde se desplegaron todos los microservicios del sistema, organizando los entornos mediante namespaces para desarrollo, pruebas y producción. Se configuraron componentes fundamentales del núcleo de la plataforma, como Cloud Config Server, Service Discovery y Zipkin, asegurando configuración centralizada, registro de servicios y trazabilidad distribuida.

En el ámbito de red y seguridad, se habilitaron los servicios esenciales de Kubernetes, se configuró un Ingress Controller con TLS/HTTPS, se aplicaron políticas de red y ServiceAccounts con permisos mínimos, y se realizaron análisis de vulnerabilidades de las imágenes. La gestión de configuración se centralizó mediante ConfigMaps, mientras que las credenciales y datos sensibles se manejaron con Secrets, incluyendo rotación automática y, opcionalmente, SealedSecrets para reforzar la seguridad.

El despliegue de la plataforma se apoyó en un pipeline CI/CD completo mediante GitHub Actions, incorporando pruebas automatizadas, estrategias de despliegue como Canary, mecanismos de rollback y gestión de microservicios a través de Helm Charts. Se implementó la persistencia de datos con Persistent Volumes y Persistent Volume Claims, junto con estrategias de copia de seguridad y restauración para herramientas críticas como Prometheus y Grafana.

Finalmente, se integraron componentes de observabilidad y monitoreo (Prometheus, Grafana y Zipkin) y mecanismos de autoscaling dinámico mediante HPA y KEDA, complementados con pruebas de carga y rendimiento para garantizar la estabilidad del sistema bajo condiciones de tráfico variables. Este conjunto de prácticas permitió consolidar una plataforma robusta, escalable y alineada con los estándares modernos de arquitectura cloud-native.

Requerimientos del proyecto

1. Arquitectura e Infraestructura (15%)

Debes construir toda la arquitectura de microservicios en Kubernetes:

- Crear y configurar un clúster Kubernetes (Minikube, Kind o cloud).
- Implementar todos los microservicios del repositorio base dentro del clúster.
- Documentar la arquitectura con diagramas completos.
- Crear namespaces separados para los ambientes (dev, qa, prod).
- Configurar correctamente las dependencias entre servicios, respetando la arquitectura original.

2. Red y Seguridad (15%)

- Configurar los servicios de Kubernetes (ClusterIP, NodePort).
- Implementar un Ingress Controller para exponer la API Gateway.
- Configurar NetworkPolicies para restringir comunicación entre servicios.
- Habilitar TLS/HTTPS para los endpoints públicos.
- Crear ServiceAccounts con permisos mínimos (RBAC).
- Realizar escaneo de imágenes para detectar vulnerabilidades.
- Implementar seguridad a nivel de pod siguiendo Pod Security Standards.

3. Gestión de Configuración y Secretos (10%)

Gestionar todas las configuraciones y datos sensibles:

- Migrar las configuraciones de Spring Boot a ConfigMaps.
- Implementar Secrets para credenciales y datos sensibles.
- Configurar rotación de secretos.
- Usar variables de entorno o volúmenes para inyectar configuraciones.
- Centralizar toda la configuración (puede usarse un servicio cloud).
- (Opcional) Implementar Sealed Secrets o HashiCorp Vault.

4. Estrategias de Despliegue y CI/CD (15%)

- Pipeline de CI/CD completo con Jenkins, GitLab CI o GitHub Actions para todos los microservicios.
- Implementación de Canary Deployments para los servicios orientados al cliente.
- Implementación de Blue-Green Deployments para servicios críticos.
- Configuración de pruebas automatizadas como gate para los despliegues.
- Estrategia de rollback automatizado ante fallos.
- Uso de Helm Charts para empaquetar los microservicios Implementación de dependencias correctas en el orden de despliegue.

5. Almacenamiento y Persistencia (10%)

- Implementación de Persistent Volumes y Persistent Volume Claims para las bases de datos.
- Configuración de StorageClass adecuada para el entorno Backup y restauración de datos persistentes.
- Implementación de bases de datos con replicación (opcional).
- Estrategia para migraciones de esquemas de base de datos Gestión adecuada del estado para servicios stateful.

6. Observabilidad y Monitoreo (15%)

- Implementación completa de Prometheus+Grafana.
- Aprovechamiento de los Actuator endpoints de Spring Boot para monitoreo.

- Configuración de alertas para situaciones críticas Sistema de logging centralizado.
- ELK, Loki, o similar) Tracing distribuido con Jaeger o similar (integrando con Zipkin que ya está en la arquitectura).
- Instrumentación de aplicaciones para exponer métricas Dashboards personalizados para diferentes stakeholders (técnicos y de negocio).
- Monitoreo de las comunicaciones entre servicios.

7. Autoscaling y Pruebas de Rendimiento (10%)

- Implementación de Horizontal Pod Autoscaler para todos los microservicios
- Implementación de KEDA para escalado basado en eventos
- Métricas personalizadas para autoscaling (no solo CPU/memoria)
- Pruebas de estrés con JMeter, Locust o similar
- Análisis de resultados y optimización de recursos Implementación de Quality of Service Pruebas de carga que simulen escenarios de uso real de la aplicación e commerce.

Estructura del proyecto:

1. Helm-charts

Esta carpeta contiene todos los Helm Charts de los microservicios del sistema, organizados en dos grupos:

Core:

Incluye los charts de los servicios fundamentales: cloud-config, service-discovery y zipkin. Estos servicios constituyen la base del ecosistema y deben estar desplegados antes de cualquier otro servicio, ya que proporcionan configuración centralizada, descubrimiento de servicios y trazabilidad de requests.

Service:

Contiene los charts de los microservicios adicionales, los cuales dependen de los servicios Core o extienden su funcionalidad.

2. Infrastructure

Esta carpeta agrupa todos los recursos y configuraciones previas al despliegue de los microservicios, tales como:

- ConfigMaps: gestión de configuraciones no sensibles para los servicios, como la centralización de parámetros comunes utilizados por todos los microservicios.
- Secrets: almacenamiento seguro de credenciales y datos sensibles, por ejemplo, credenciales de Git o de Actuator.
- TLS: certificados y claves TLS, con soporte para secretos sellados mediante SealedSecrets.
- CronJobs: tareas programadas de mantenimiento y operación, como la rotación periódica de los secretos.
- Recursos de seguridad: políticas de acceso, roles y bindings, incluyendo roles comunes y roles específicos que permiten la rotación de secretos.
- Configuración de Prometheus: definición de alertas, reglas y targets para el monitoreo de los servicios.
- Namespaces: entornos aislados para dev, qa y prod.
- Prometheus: archivos de configuración personalizada.

3. Scripts

Contiene automatizaciones y utilidades de soporte para la operación del sistema:

- Escaneo de vulnerabilidades: análisis de seguridad en imágenes de los contenedores.
- Auditoría de seguridad de Pods: generación de reportes del securityContext de cada Pod, validando permisos y configuraciones críticas.
- Rotación de Secrets TLS: scripts automatizados para actualizar certificados y claves.
- Análisis histórico de vulnerabilidades: registro de auditorías y reportes previos, organizado en una subcarpeta específica.

Nota: el parámetro `readOnlyRootFilesystem` se desactivó en algunos microservicios, dado que requerían permisos de escritura en disco. Para despliegues futuros, se recomienda montar volúmenes específicos para escritura, manteniendo el filesystem raíz en modo solo lectura. Probablemente tener en cuenta en los entornos de qa y producción.

Arquitectura del sistema:

Arquitectura original: El sistema estaba diseñado para ejecutarse de manera centralizada mediante Docker Compose, sin separación por namespaces ni gestión centralizada de configuraciones. Cada microservicio incluía sus propias configuraciones y puertos, definidos localmente en el archivo de configuración sin utilizar ConfigMaps ni Secrets para centralizar parámetros o credenciales.

Arquitectura actual:

- **Microservicios:** cada servicio es independiente, con su propio Helm Chart y configuración.
- **Servicios de core:** cloud-config (gestión de configuraciones), service-discovery (registro y descubrimiento de servicios), zipkin (trazabilidad distribuida).
- **Servicios de negocio:** aquellos servicios secundarios relacionados con pagos, ordenes, favoritos, envíos, productos y demás.
- **Infraestructura compartida:** ConfigMaps, Secrets, TLS, CronJobs, políticas de seguridad. Con el fin de permitir la centralización de configuraciones no sensibles para todos los microservicios, el almacenamiento seguro de credenciales y datos sensibles, definición de replicas, y posibilidad del escalamiento del sistema si así se requiere.
- **Monitoreo:** Prometheus para métricas y alertas, Grafana para dashboards visuales, Zipkin para trazabilidad distribuida.
- **Namespaces por entorno:** dev, qa y prod.

El despliegue de la plataforma se realizó completamente mediante Helm Charts, ajustando los valores de cada servicio según sus necesidades de puerto, recursos y criticidad. Los microservicios principales del sistema incluyen:

- Api gateway.
- Shipping service.
- Favourite service.
- Payments service.
- Order service.
- Product service.
- Proxy client.
- User service.

Cada microservicio contaba con un puerto específico asignado. La mayoría se configuró como ClusterIP, según su función dentro del ecosistema. Por ejemplo, para api gateway se habilitó un Ingress con el dominio *ecommerce.local*, ya que todos los servicios debían ser accesibles a través de este punto de entrada centralizado.

Gestión de recursos:

Los servicios críticos como cloud-config, service-discovery y api-gateway recibieron mayor asignación de recursos, debido a su nivel de criticidad dentro del ecosistema.

El resto de los microservicios se desplegaron con recursos más bajos, considerando que el entorno utilizado fue local (Minikube), con limitaciones de CPU y memoria.

Durante las pruebas y validación, se ajustaron los recursos de manera incremental para garantizar que cada servicio pudiera mantenerse operativo sin afectar al cluster.

Se definió un ConfigMap denominado *common-env*, que contenía configuraciones compartidas por todos los microservicios, tales como:

- Conexión a Zipkin para trazabilidad.
- Conexión a Eureka / Service Discovery.
- Parámetros de configuración centralizada desde Cloud Config.

De modo que, los microservicios no dependieran de configuraciones locales manteniendo consistencia y facilidad de mantenimiento.

Secrets, rotación y certificados:

Es importante destacar que los Secrets no se encuentran presentes en ningún archivo del repositorio, sino que fueron creados directamente mediante la consola y asignados según los recursos correspondientes. Por ejemplo:

- El repositorio de Cloud Config Server es privado y contiene todas las configuraciones centrales. Para acceder a él, se creó un Secret con las credenciales necesarias, incluyendo usuario y token.
- Para Actuador, se utilizó en dos servicios de prueba: Order Service y Payment Service. Dado que toda la configuración está gestionada mediante Helm, los deployments simplemente referencian estos Secrets.

Toda la configuración específica de cada microservicio se define en el archivo values.yaml correspondiente de cada Helm Chart.

El acceso a los servicios se realiza a través de API Gateway, para lo cual se configuró un certificado TLS auto firmado. Este certificado no es avalado por ninguna autoridad certificadora, ya que se utiliza únicamente para pruebas en entornos locales. Debido a que HTTPS requiere TLS/SSL y espera un certificado confiable, las consultas se realizan usando:

```
curl -k https://ecommerce.local/<nombre-del-servicio>/api/<funcionalidad>
```

Por ejemplo:

```
curl -k https://ecommerce.local/product-service/api/products
```

Para la configuración del dominio en Minikube, se obtiene la IP del cluster mediante:

```
minikube ip
```

Posteriormente, se añade una entrada en el archivo /etc/hosts, asociando la IP obtenida con el dominio ecommerce.local, para que el sistema reconozca el nombre de dominio y pueda enrutar correctamente las solicitudes al API Gateway.

Además, es importante destacar que la consulta se realiza con la opción -k porque se está trabajando sobre HTTPS con un certificado autofirmado. Esta opción permite aceptar dicho certificado, aunque no sea considerado seguro.

```
lina@Lina:~/kubernetes/ecommerce-microservice-backend-app/helm-charts$ curl -k https://ecommerce.local/user-service/api/users
{
  "collection": [ {
    "userId": 1,
    "firstName": "selim",
    "lastName": "horri",
    "imageUrl": "https://bootdey.com/img/Content/avatar/avatar7.png",
    "email": "springxyzabcboot@gmail.com",
    "phone": "+21622125144",
    "credential": {
      "credentialId": 1,
      "username": "selimhorri",
      "password": "$2a$04$/S7cwjHPZul03sPEivycweKTBvLyjYdaRwmeaFbiqKy9es/3w4Q0B6",
      "roleBasedAuthority": "ROLE_USER",
      "isEnabled": true,
      "isAccountNonExpired": true,
      "isAccountNonLocked": true,
      "isCredentialsNonExpired": true
    }
  }, {
    "userId": 2,
    "firstName": "amine",
    "lastName": "ladjimi",
    "imageUrl": "https://bootdey.com/img/Content/avatar/avatar7.png",
    "email": "springxyzabcboot@gmail.com",
    "phone": "+21622125144",
    "credential": {
      "credentialId": 2,
      "username": "amineladjimi",
      "password": "$2a$04$8D80uqPbE4LhRckvtBAHrOmpelwE92xNNVtyK8Z/1rJFjsImpjBkm",
      "roleBasedAuthority": "ROLE_USER",
      "isEnabled": true,
      "isAccountNonExpired": true,
      "isAccountNonLocked": true,
      "isCredentialsNonExpired": true
    }
  }, {
    "userId": 3,
    "firstName": "omar",
    "lastName": "derouiche",
    "imageUrl": "https://bootdey.com/img/Content/avatar/avatar7.png",
    "email": "springxyzabcboot@gmail.com",
    "phone": "+21622125144",
    "credential": {
      "credentialId": 3,
      "username": "omarderouiche",

```

Ilustración 1. Validación del endpoint usando api-gateway.

Si bien el certificado TLS utilizado es auto firmado, contiene datos sensibles debido a la clave privada que genera. Por este motivo, se creó un Secret que posteriormente se almacenó como un SealedSecret, garantizando mayor seguridad y control en el manejo del certificado.

Además, es importante tener en cuenta que, los certificados auto firmados están presentes solo en entornos de desarrollo. En producción se recomienda usar cert-manager con Let's Encrypt.

La rotación de este certificado no está automatizada mediante un CronJob, sino que se realiza ejecutando un script manual. El procedimiento del script es el siguiente:

- Se genera un nuevo certificado TLS.
- Se crea un Secret temporal basado en dicho certificado.
- Se utiliza kubeseal para generar el SealedSecret correspondiente.
- El SealedSecret actualizado se guarda en la carpeta infrastructure.

Por otro lado, los Secrets de Actuator sí cuentan con un CronJob programado, que se ejecuta cada 90 días (el día 1 de cada trimestre a las 2:00 AM). Durante esta rotación automática:

- La nueva clave se genera de manera aleatoria (random).
- El usuario se mantiene como Admin.

Este esquema garantiza que tanto los certificados TLS como las credenciales sensibles se gestionen de forma segura y controlada, respetando las políticas de rotación y acceso centralizado.

Configuración centralizada:

Ahora bien, como se mencionó anteriormente, la configuración se encuentra centralizada mediante Cloud Config Server para garantizar un mayor control sobre los parámetros de los microservicios. Inicialmente, la imagen de Cloud Config Server apuntaba a un repositorio que no contenía todas las configuraciones necesarias para los servicios. Por esta razón, se partió de dicho repositorio para crear uno nuevo con la configuración específica para el entorno de desarrollo. Este proceso se realizó validando las configuraciones locales de cada microservicio según su entorno original, ya fuera stage, prod o dev. Con base en esta revisión, se generaron las configuraciones para el entorno de desarrollo, utilizando una base de datos local H2, adecuada únicamente para entornos de pruebas efímeros, garantizando consistencia en desarrollo sin afectar otros entornos. De manera similar, para habilitar configuraciones de stage o prod, basta con validar las configuraciones locales de esos entornos y asignar la base de datos correspondiente.

Dado que la imagen original de Cloud Config Server apuntaba al repositorio incompleto, se actualizó la referencia al nuevo repositorio, se generó nuevamente el JAR con todas las configuraciones y se subió a un Docker Hub. En consecuencia, en el archivo values.yaml del Helm Chart se actualizó la referencia a la nueva imagen, garantizando que los microservicios consuman la configuración centralizada más reciente y adecuada para su entorno.

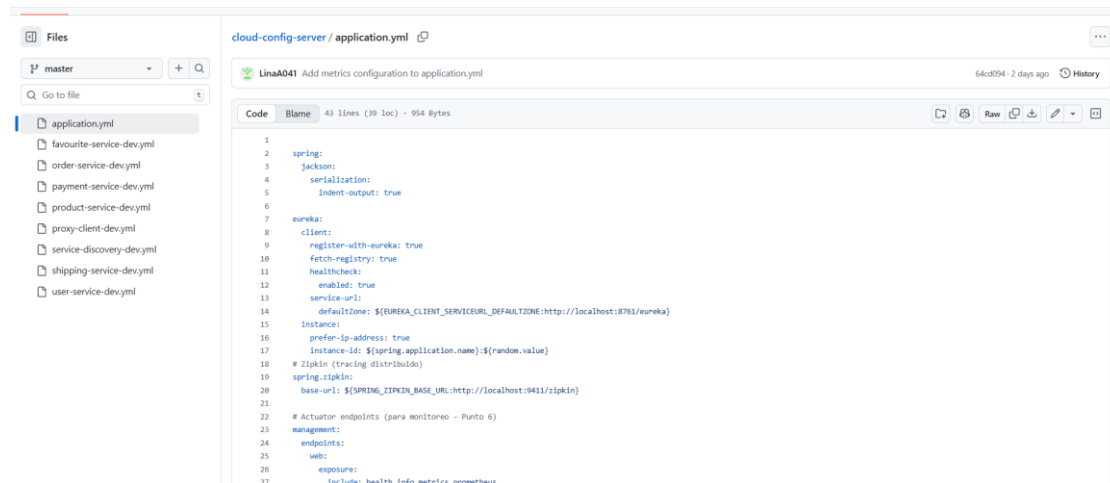


Ilustración 2. Repositorio con configuraciones para dev.

Tal como se puede observar en la imagen a continuación, las configuraciones de los microservicios se están obteniendo directamente desde Cloud Config Server, confirmando que la centralización y gestión de parámetros se encuentran operativas.

```
lina@Lina:~/kubernetes/e-commerce-microservice-backend-app$ curl -s http://localhost:9296/favourite-service/dev | jq .
{
  "name": "favourite-service",
  "profiles": [
    "dev"
  ],
  "label": null,
  "version": "93239d186a2019443a181a0a04a33f7b9c8ba0dd",
  "state": null,
  "propertySources": [
    {
      "name": "https://github.com/LinaA041/cloud-config-server/favourite-service.yml",
      "source": {
        "server.port": 8800,
        "server.servlet.context-path": "/favourite-service",
        "spring.jpa.show-sql": true,
        "spring.jpa.hibernate.ddl-auto": "validate",
        "spring.jpa.properties.hibernate.dialect": "${SPRING_JPA_DATABASE_PLATFORM:org.hibernate.dialect.H2Dialect}",
        "spring.jpa.properties.hibernate.use_sql_comments": true,
        "spring.jpa.properties.hibernate.format_sql": true,
        "spring.h2.console.enabled": true,
        "spring.h2.console.path": "/h2-console",
        "resilience4j.circuitbreaker.instances.favouriteService.register-health-indicator": true,
        "resilience4j.circuitbreaker.instances.favouriteService.failure-rate-threshold": 50,
        "resilience4j.circuitbreaker.instances.favouriteService.minimum-number-of-calls": 5,
        "resilience4j.circuitbreaker.instances.favouriteService.wait-duration-in-open-state": "5s",
        "resilience4j.circuitbreaker.instances.favouriteService.sliding-window-size": 10,
        "logging.level.org.hibernate.SQL": "DEBUG",
        "logging.level.org.springframework.web": "DEBUG"
      }
    },
    {
      "name": "https://github.com/LinaA041/cloud-config-server/application.yml",
      "source": {
        "spring.jackson.serialization.indent-output": true,
        "eureka.client.register-with-eureka": true,
        "eureka.client.fetch-registry": true,
        "eureka.client.service-url.defaultZone": "${EUREKA_CLIENT_SERVICEURL_DEFAULTZONE:http://localhost:8761/eureka}",
        "eureka.instance.prefer-in-address": true
      }
    }
  ]
}
```

Ilustración 3. Configuración de favourite Service para el entorno de dev.

Orden de despliegue, análisis de vulnerabilidades y rbac:

Además, dentro de los mismos *Deployments* se implementó un contenedor BusyBox encargado de esperar hasta que Cloud Config Server y Service Discovery (Eureka) estén completamente disponibles antes de iniciar los microservicios restantes. Con esto se garantiza que cada servicio cargue correctamente su configuración desde el Cloud Config Server y que Eureka esté listo para registrar las instancias sin errores al arrancar.

Adicionalmente, se utilizó Trivy para el escaneo de vulnerabilidades en las imágenes, y se aplicaron políticas de seguridad como el Pod Security Group. También se creó la configuración de RBAC, en la cual se definió un *ServiceAccount* llamado *microservice-sa* junto con un *Role* que otorga permisos para visualizar, obtener y listar recursos como ConfigMaps, Secrets, Endpoints y Services.

Adicionalmente, debido a la implementación de la rotación de secretos, fue necesario definir un *Role* más avanzado, con permisos completos sobre Secrets para poder crearlos, actualizarlos, rotarlos y eliminarlos. Este rol fue asignado a los CronJobs, pues incluye permisos para ejecutar *rollout restart* sobre Deployments, consultar Pods y Jobs, y gestionar SealedSecrets. Esto permite que el proceso de rotación se ejecute de forma segura y automatizada dentro del entorno Kubernetes.

Para la rotación de secretos mediante CronJobs, es importante tener en cuenta que el secreto de Git depende de la obtención de un token con los permisos necesarios para acceder al repositorio.

```
lina@lina:~/kubernetes/ecommerce-microservice-backend-app$ kubectl get cronjobs -n dev
NAME                                SCHEDULE    TIMEZONE    SUSPEND    ACTIVE    LAST SCHEDULE    AGE
rotate-actuator-secret             0 2 1 */3 *  <none>     False      0           <none>     149m
rotate-git-secret                  0 3 1 */6 *  <none>     True       0           <none>     149m
lina@lina:~/kubernetes/ecommerce-microservice-backend-app$
```

Ilustración 4. Cronjobs del sistema.

A continuación, se muestra un fragmento representativo de los resultados del escaneo de vulnerabilidades realizado sobre las imágenes de los distintos microservicios. Este análisis se

efectuó para evaluar posibles riesgos de seguridad presentes en las imágenes utilizadas en el despliegue.

kubernetes / ecommerce-microservice-backend-app / scripts / security-reports / images / api-gateway- / 195b4 / ct4-80bgz_selimhorri_api-gateway-ecommerce-boot_u.1.u.txt

report summary

sealedAa ab * No results ↑ ↓ ≡ ×

Target	Type	Vulnerabilities	Secrets
selimhorri/api-gateway-ecommerce-boot:0.1.0 (debian 11.2)	debian	761	-
home/app/.mvn/wrapper/maven-wrapper.jar	jar	0	-
home/app/api-gateway.jar	jar	86	-
home/app/target/api-gateway-v0.1.0.jar	jar	86	-

Legend:

- '-': Not scanned

- '0': Clean (no security findings detected)

selimhorri/api-gateway-ecommerce-boot:0.1.0 (debian 11.2)

=====

Total: 761 (UNKNOWN: 10, LOW: 210, MEDIUM: 273, HIGH: 209, CRITICAL: 59)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	
apt	CVE-2011-3374	LOW	affected	2.2.4		It was found that apt- correctly... https://avd.aquasec.com
bash	CVE-2022-3715	HIGH		5.1-2+b3		bash: a heap-buffer-ovi https://avd.aquasec.com

Ilustración 5. Reporte parcial del análisis de vulnerabilidades (api-gateway).

Además, el diagrama que se presenta a continuación ilustra cómo se encuentra diseñada la arquitectura en este entorno local. Se incluyen los microservicios desplegados (Product Service, User Service, Order Service, Shipping Service, Favourite Service y Payment Service), junto con los servicios de infraestructura críticos como Cloud Config Server, Service Discovery (Eureka) y API Gateway. También se representan los sistemas de observabilidad, incluyendo Prometheus y Grafana, que permiten la recolección de métricas, monitoreo de rendimiento y visualización de datos. La arquitectura refleja un entorno de desarrollo local con Minikube, donde los componentes se comunican mediante servicios internos del clúster, y se manejan configuraciones, escalabilidad y persistencia de manera centralizada.

Para un diagrama más detallado consulte el enlace proporcionado en la nota al pie¹.

¹ Diagrama de arquitectura: <https://www.figma.com/board/EEUBzv8aVnggiUEHXQJWdO/Arquitectura-General-Microservicios-Local?node-id=0-1&t=vI3bMx7oyOUOboar-1>

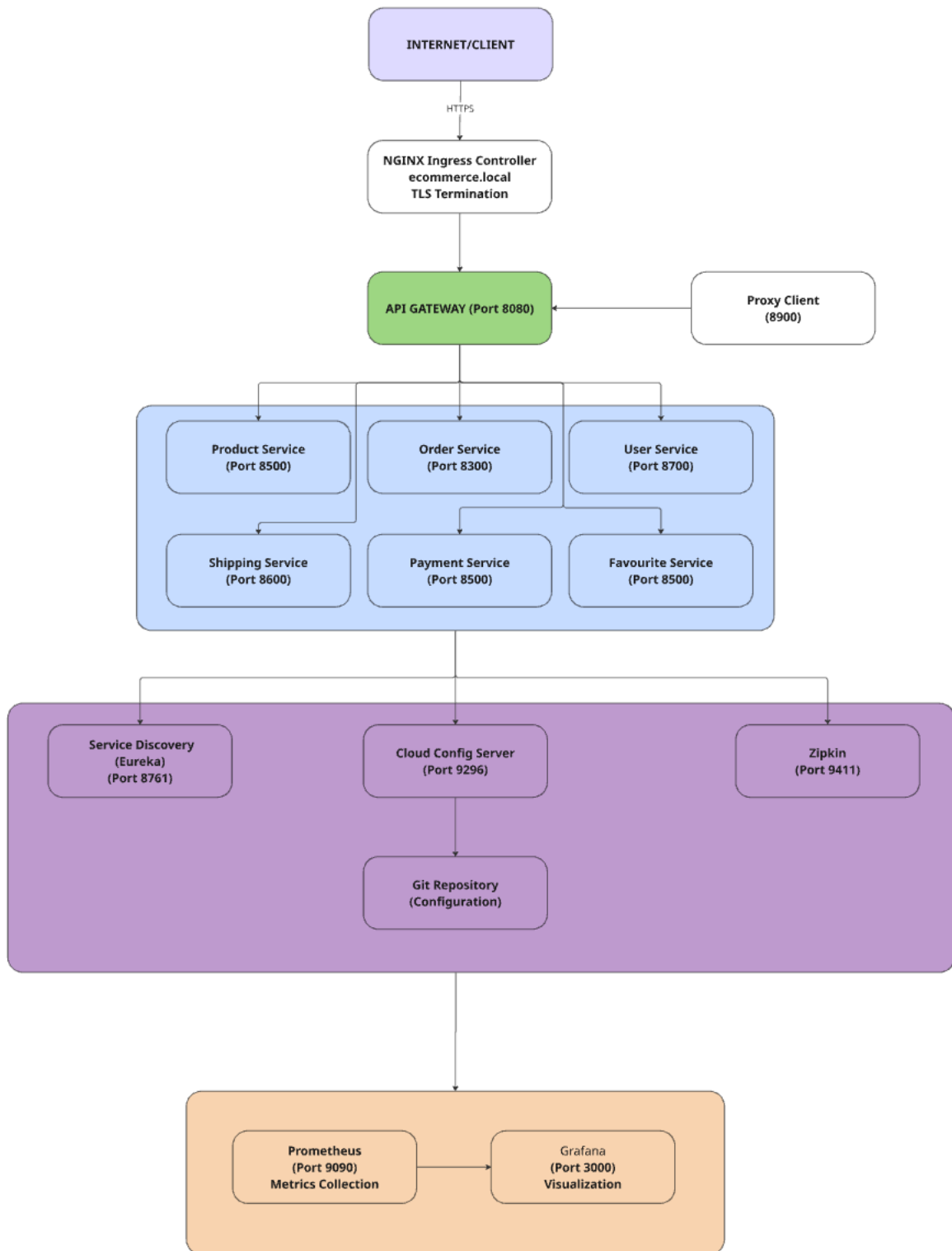


Ilustración 6. Arquitectura del sistema.

CI/CD

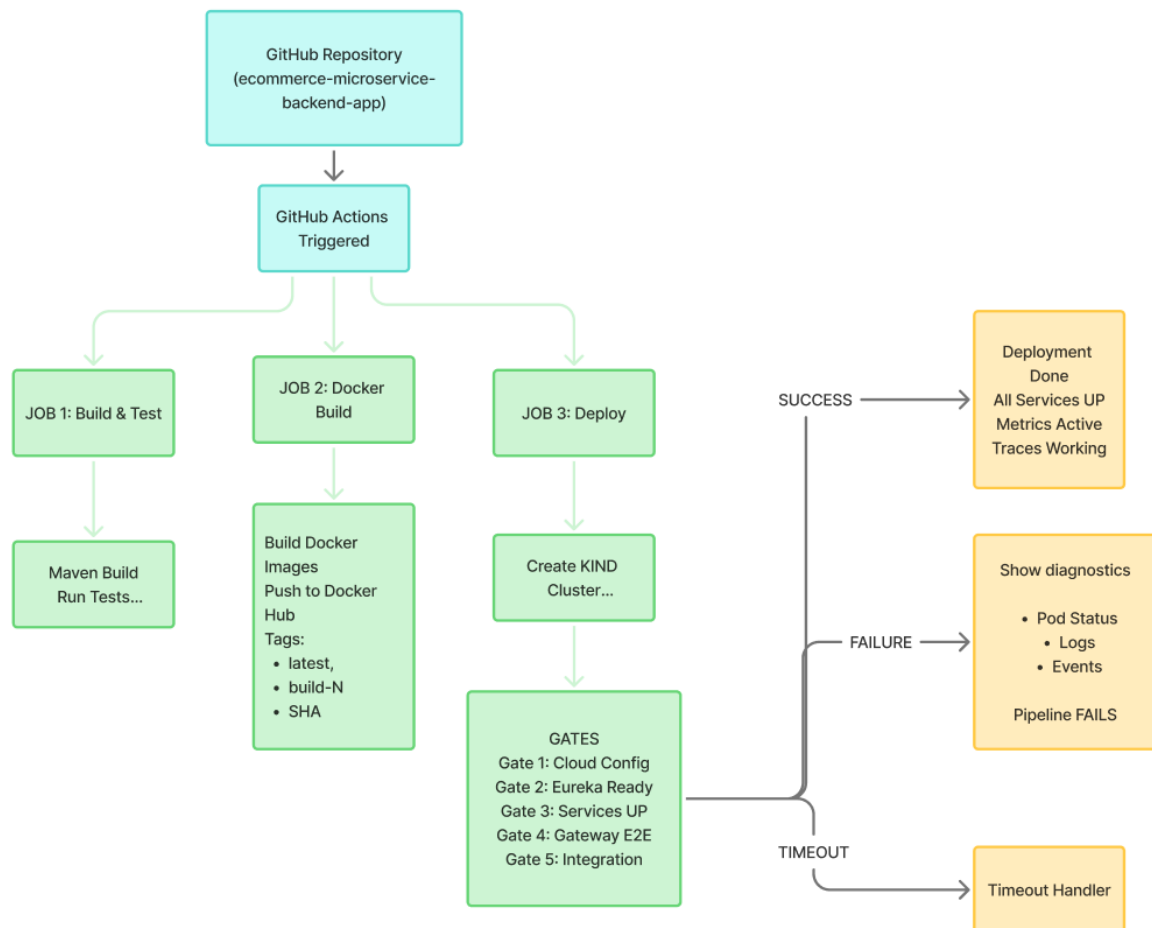


Ilustración 7. Pipeline para CI/CD.

El pipeline implementado para los microservicios del proyecto de e-commerce sigue una estrategia de integración y entrega continua completa, diseñada para garantizar la calidad, trazabilidad y despliegue confiable en entornos de desarrollo basados en Kubernetes (KIND). El flujo principal consta de tres etapas:

1. Build & Test

Objetivo: Compilar y validar cada microservicio individualmente.

Servicios incluidos: Product, User, Payment, Order, Shipping, Favourite, Proxy Client, API Gateway, Cloud Config y Service Discovery.

Acciones principales:

- Se realiza checkout del código y se configura el JDK 17.

- Compilación con Maven (mvn clean package -DskipTests) y ejecución de pruebas unitarias (mvn test).
- Generación de artefactos .jar que se almacenan temporalmente como artefactos de GitHub Actions para su uso en etapas posteriores.

2. Docker Build & Push

Objetivo: Construir imágenes Docker de cada microservicio y subirlas al registro de Docker Hub.

Acciones principales:

- Descarga de artefactos generados en la etapa anterior.
- Validación de existencia de los .jar.
- Construcción de imágenes Docker etiquetadas con latest, número de build y SHA corto del commit.
- Uso de cache de construcción para optimizar tiempos de build.
- Login seguro en Docker Hub usando secretos (DOCKER_USERNAME y DOCKER_PASSWORD).
- Push de las imágenes al registro.

3. Deploy to Kubernetes (KIND)

Objetivo: Desplegar los microservicios en un clúster local Kubernetes y validar su funcionamiento.

Acciones principales:

Preparación del clúster

- Creación de clúster KIND.
- Configuración de kubectl y Helm.
- Creación de namespace dev y secrets necesarios para acceso a repositorios y servicios.
- Configuración de ServiceAccount y RBAC para permisos adecuados.

Despliegue de servicios core

- Cloud Config y Service Discovery mediante Helm.

- Validación mediante pruebas de smoke test y verificación de endpoints /actuator/health y /actuator/info.

Despliegue de microservicios de negocio

- Product, User, Payment, Order, Shipping, Favourite y Proxy Client.
- Verificación de salud de cada servicio mediante endpoint /actuator/health.
- Registro de servicios en Eureka.

Despliegue de API Gateway

- Validación de rutas hacia los microservicios a través del gateway.

Gates de integración

- Se implementan múltiples gates para asegurar la correcta inicialización y registro de servicios antes de avanzar a la siguiente etapa.

Backup y rollback

- Creación de una copia de seguridad del deployment exitoso como artefacto (deployment-backup) con información de tags de imágenes.

En caso de falla en el despliegue, se busca realizar el rollback automático utilizando la copia de seguridad previo para restaurar el estado del clúster.

Pruebas y validación

Los endpoints de cada microservicio y del API Gateway se verifican mediante health checks automáticos. De modo, que se asegura la correcta integración con Service Discovery. Y se mantiene un flujo de despliegue seguro mediante gates y rollback en caso de errores.

Canary:

Implementación

1. Creación de Deployment con estrategia Canary

Se utiliza Helm para desplegar el API Gateway con un Deployment que soporte actualizaciones progresivas.

Se definen réplicas iniciales reducidas para la versión canaria (replicas: 1) mientras que la versión estable continúa atendiendo la mayor parte del tráfico.

Uso de labels y selectors para diferenciar entre la versión canaria y la versión estable.

2. Ruteo de tráfico

Se implementa una estrategia de ruteo progresivo, donde un porcentaje del tráfico se dirige a la versión canaria (10%). Esto se puede lograr mediante:

- Anotaciones en servicios de Kubernetes con control de tráfico (ej. [nginx.ingress.kubernetes.io/canary](https://ingress.kubernetes.io/canary)).
- Configuración en Ingress Controller para redirigir un porcentaje específico de peticiones.

3. Validación automática

La versión canaria pasa por los mismos health checks y pruebas de integración que la versión estable:

- Endpoints /actuator/health.
- Pruebas de rutas a microservicios vía API Gateway.

Se monitorean métricas en Prometheus y Grafana para detectar errores o degradación del servicio.

4. Promoción o rollback

Se debe tener en cuenta, si la versión canaria se comporta correctamente durante el periodo de prueba:

Se escala gradualmente hasta reemplazar la versión estable. Si se detectan fallos:

- Se ejecuta un rollback automático, preservando la versión estable y evitando interrupciones al tráfico de producción.

Persistencia:

En este proyecto se asegura la persistencia de datos para los servicios que requieren mantener información de manera estable, aun ante reinicios o despliegues. Aunque no se utilizan bases de datos tradicionales, se aprovechó la persistencia de Grafana y Prometheus para ilustrar la estrategia.

Persistent Volumes (PV) y Persistent Volume Claims (PVC)

Se definieron PVCs para los servicios de monitoreo:

- grafana → 4Gi, modo RWO.
- prometheus-server → 8Gi, modo RWO.
- prometheus-alertmanager → 2Gi, modo RWO.

Estos PVCs están **bound** a los PVs correspondientes, garantizando que los datos de dashboards, métricas y alertas se mantengan, aunque los pods se reinicien. Por otro lado, la clase de almacenamiento utilizada fue standard (StorageClass predeterminada en Minikube).

Backup de datos persistentes

- Para Grafana, se implementó un mecanismo de backup utilizando kubectl cp:

kubectl cp monitoring/grafana-<pod>:/var/lib/grafana ./grafana-backup

Esto copia todo el directorio de datos de Grafana a una ubicación local (grafana-backup).E incluye dashboards, paneles, fuentes de datos y configuraciones guardadas.

Para restaurar:

- Detener el pod de Grafana.
- Copiar el contenido de grafana-backup al directorio /var/lib/grafana dentro del pod o PV.
- Reiniciar el pod para que Grafana cargue los datos restaurados.

Estrategia de replicación y resiliencia

Aunque Minikube no soporta replicación de PVs de forma nativa, se garantiza que:

Los datos críticos se mantienen en almacenamiento persistente (PVC bound to PV). Además, la copia de seguridad manual permite restaurar el estado previo en caso de pérdida o error.

Se podrían extender estas prácticas a bases de datos futuras mediante:

- Creación de PVCs para cada servicio con estado.
- Configuración de jobs de backup y restauración automatizados.
- Uso de StorageClass y estrategias de replicación en entornos productivos.

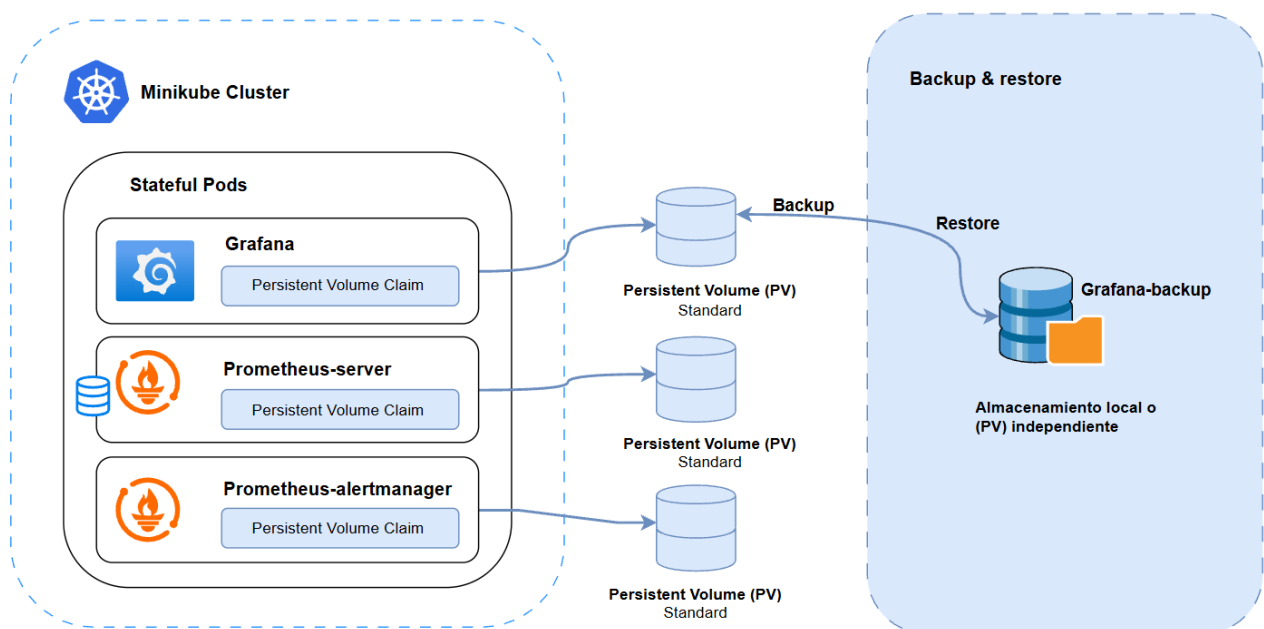


Ilustración 8. Diagrama ilustrativo de persistencia.

El diagrama muestra una arquitectura de aplicaciones stateful en Kubernetes donde cada componente —Grafana, Prometheus Server y Prometheus Alertmanager— utiliza su propio volumen persistente para almacenar datos críticos. Para garantizar la disponibilidad y recuperación de la información, se implementa un sistema de backup y restore que sigue un flujo estructurado: los datos de Grafana se respaldan periódicamente en un volumen de almacenamiento dedicado, conservando dashboards, configuraciones y datos de paneles, aunque estos también pueden ser almacenados de manera local. Cuando es necesario recuperar la información, el proceso de restore despliega una nueva instancia de Grafana con un volumen limpio y restaura en él los datos desde el respaldo, asegurando así la integridad y continuidad del servicio sin afectar los volúmenes de las demás aplicaciones.

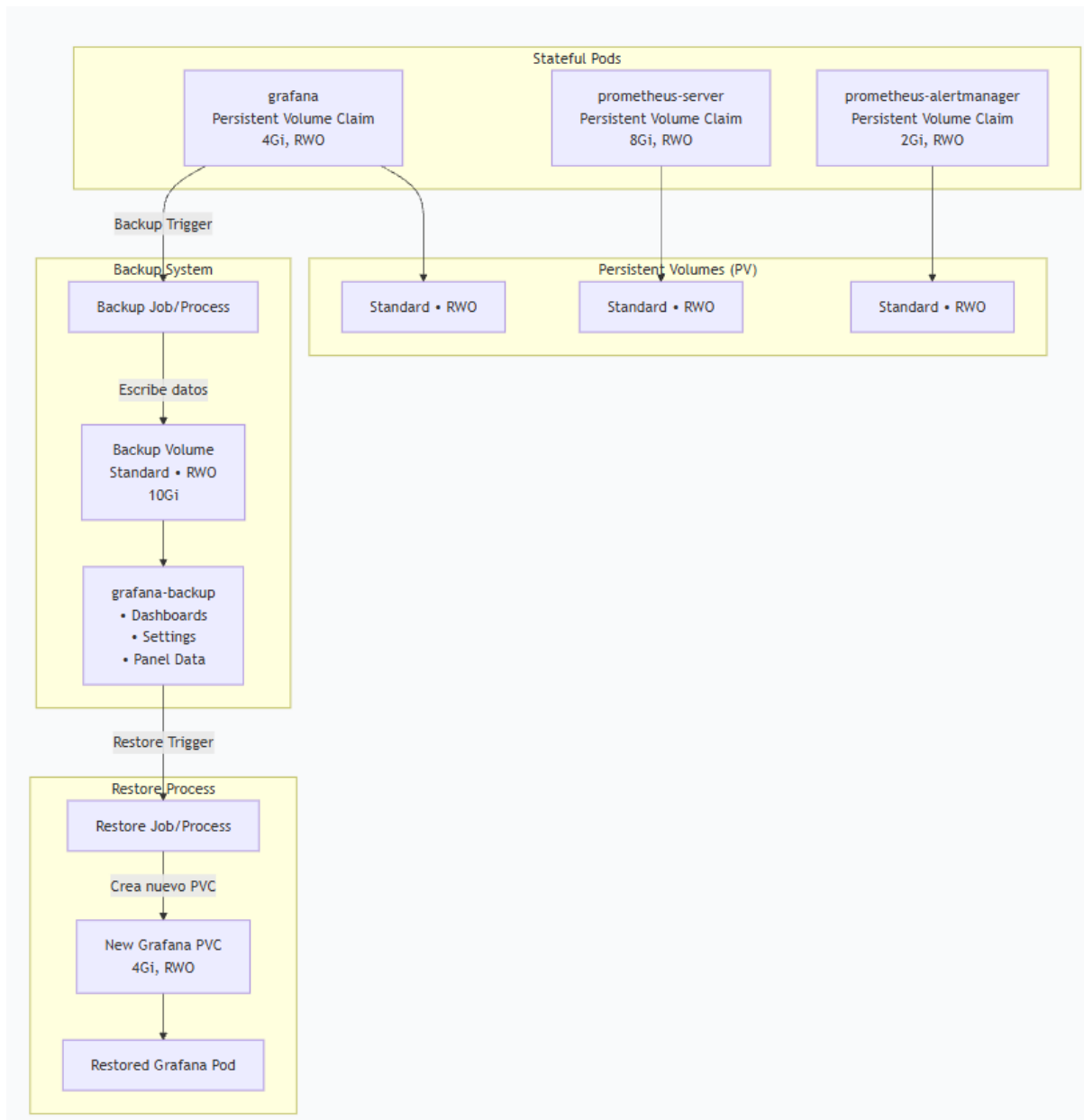


Ilustración 9. Comportamiento esperado de backup.

Monitoreo:

El sistema de monitoreo de la plataforma se implementa mediante Prometheus y Grafana, utilizando Helm charts para la instalación en Kubernetes. De modo que, se permite:

- Recopilar métricas de los microservicios.
- Configurar alertas personalizadas.
- Crear dashboards propios que muestran métricas clave, como solicitudes por minuto y total de requests.

- Persistir la configuración de dashboards a través de un Persistent Volume Claim (PVC) en Grafana.
-

Dentro del directorio de infrastructure y en el subdirectorio de prometheus se encuentran los siguientes archivos:

Archivo	Descripción
grafana-values.yml	Valores personalizados para Grafana, incluyendo PVC para persistencia de dashboards.
metrics-values.yml	Valores personalizados para Prometheus, incluyendo configuración de scrapers y métricas adicionales.
prometheus-alerts.yml	Definición de alertas para Prometheus.
order-service-scaledobject.yml	Configuración de escalado automático para order-service basado en métricas de Prometheus. (Para Keda).
payment-service-scaledobject.yml	Configuración de escalado automático para payment-service (Para Keda).

Tabla 1. Archivos necesarios para la configuración de prometheus y grafana.

De modo que, para lo indicado anteriormente, se tiene que, para Prometheus se definieron valores personalizados a través del archivo metrics-values.yml, donde se configuraron los endpoints a monitorear y se añadieron métricas específicas de los microservicios críticos, como order-service y payment-service.

Grafana se desplegó con valores personalizados definidos en grafana-values.yml. Entre las configuraciones más importantes se incluyó un Persistent Volume Claim (PVC) para garantizar la persistencia de dashboards y configuraciones, incluso si el pod se reinicia. Esto permitió crear dashboards personalizados que muestran métricas clave como solicitudes por minuto de cada microservicio y el total de requests procesados. La persistencia asegura que los dashboards no se pierdan y que cualquier configuración de alertas o paneles adicionales permanezca consistente entre despliegues.

Además, se definieron alertas en prometheus-alerts.yml para supervisar el estado y rendimiento de los servicios. Estas alertas contemplan situaciones críticas como alta latencia,

errores en los microservicios y sobrecarga de CPU o memoria. Gracias a esta configuración, Prometheus no solo recopila métricas, sino que también permite reaccionar ante situaciones anómalas, integrándose incluso con sistemas de escalado automático mediante KEDA si fuera necesario.

En general, la implementación de Prometheus y Grafana se diseñó para proporcionar visibilidad completa sobre el funcionamiento de los microservicios, permitiendo tanto la monitorización en tiempo real como la generación de métricas históricas. Los dashboards personalizados y la capacidad de alertas configurables aseguran que el equipo pueda tomar decisiones basadas en datos precisos y mantener la estabilidad del sistema. Se indica, por ejemplo, la cantidad de servicios disponibles por namespace, el tiempo promedio de respuesta por servicio, además de que, se obtiene cantidad de solicitudes que un servicio recibe por minuto.

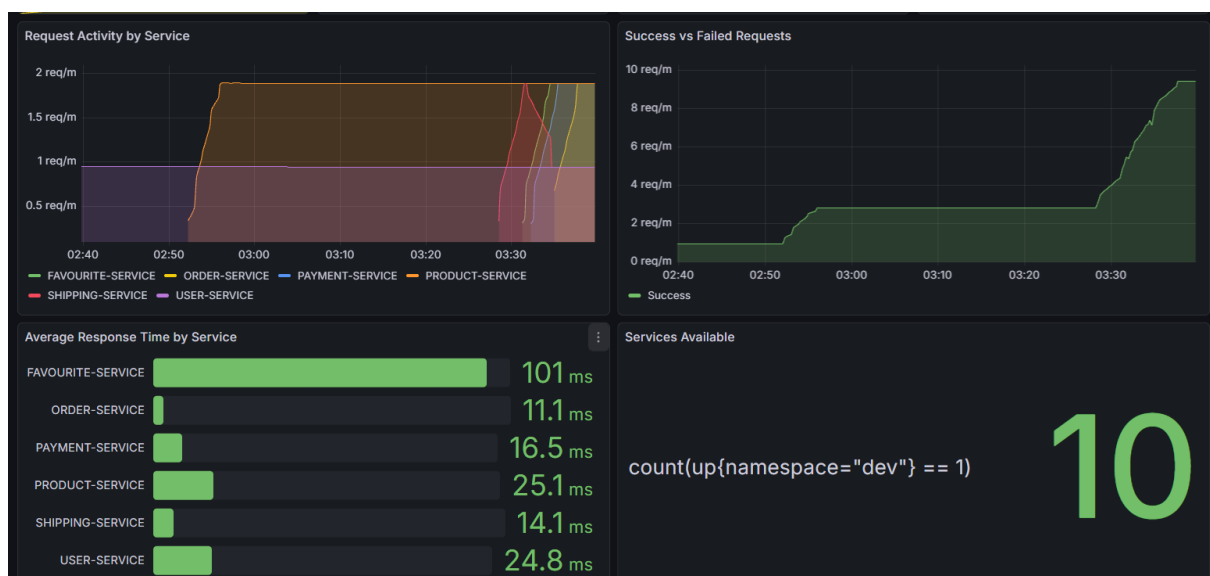


Ilustración 10. Dashboard personalizado en Grafana.



Ilustración 11. Ilustración de dashboard personalizado en Grafana.

alertname="HighVMMemory"	application="PAYMENT-SERVICE"	severity="warning"	team="backend"	FIRING	25m 28.867s	71.59262300741793
description	PAYMENT-SERVICE () está usando 71.59262300741793% de memoria heap.					
summary	Memoria heap alta en PAYMENT-SERVICE					
alertname="HighVMMemory"	application="FAVOURITE-SERVICE"	severity="warning"	team="backend"	PENDING	28.867s	50.36607222123579
description	FAVOURITE-SERVICE () está usando 50.36607222123579% de memoria heap.					
summary	Memoria heap alta en FAVOURITE-SERVICE					

Ilustración 12. Alertas para Payment y Favourite Service.

alertname="HighOrLowRequestRate"	application="FAVOURITE-SERVICE"	severity="warning"	team="backend"	FIRING	3h 8m 42.597s	2.4108042318832807
description	El servicio FAVOURITE-SERVICE está recibiendo un número inusual de solicitudes.					
summary	Actividad inusual en FAVOURITE-SERVICE					
alertname="HighOrLowRequestRate"	application="SHIPPING-SERVICE"	severity="warning"	team="backend"	FIRING	3h 42.597s	1.206058671207123
description	El servicio SHIPPING-SERVICE está recibiendo un número inusual de solicitudes.					
summary	Actividad inusual en SHIPPING-SERVICE					
alertname="HighOrLowRequestRate"	application="PAYMENT-SERVICE"	severity="warning"	team="backend"	FIRING	2h 55m 42.597s	1.2054019022660136
description	El servicio PAYMENT-SERVICE está recibiendo un número inusual de solicitudes.					
summary	Actividad inusual en PAYMENT-SERVICE					

Ilustración 13. Alertas para detección de número inusual de solicitudes.

http://10.244.0.66:9153/metrics	instance="10.244.0.66:9153"	job="kubernetes-service-endpoints"	k8s_app="k8s-dns"	kubernetes_io_cluster_service="true"	43.294s ago	8ms	UP
http://10.244.0.89:8800/favourite-service/actuator/prometheus	kubernetes_io_name="CoreDNS"	namespace="kube-system"	node="ecommerce-app"	service="k8s-dns"	53.005s ago		UP
http://10.244.0.92:8800/favourite-service/actuator/prometheus	app_kubernetes_io_instance="favourite-service"	app_kubernetes_io_managed_by="Helm"	app_kubernetes_io_name="favourite-service"	helm_sh_chart="favourite-service-1.0.0"	41.914s ago	9ms	UP
http://10.244.0.90:8400/payment-service/actuator/prometheus	app_kubernetes_io_version="0.1.0"	instance="10.244.0.89:8800"	job="kubernetes-service-endpoints"	namespace="dev"	43.673s ago	21ms	UP
	node="ecommerce-app"	service="favourite-service"					
	app_kubernetes_io_instance="payment-service"	app_kubernetes_io_managed_by="Helm"	app_kubernetes_io_name="payment-service"	helm_sh_chart="payment-service-1.0.0"			
	app_kubernetes_io_version="0.1.0"	instance="10.244.0.90:8400"	job="kubernetes-service-endpoints"	namespace="dev"			
	node="ecommerce-app"	service="payment-service"					

Ilustración 14. Visualización de los servicios en prometheus.

Auto escalamiento

En el proyecto se implementó Horizontal Pod Autoscaler (HPA) para todos los microservicios desplegados en el cluster Kubernetes. Esto permite que la cantidad de réplicas de cada microservicio se ajuste automáticamente en función de la carga de CPU y de métricas personalizadas.

Configuración de HPA

Cada microservicio cuenta con un HPA configurado con un mínimo de 1 réplica y un máximo de 2 o 3 réplicas, dependiendo de la criticidad y los recursos asignados.

Para dos microservicios clave (por ejemplo, order-service y payment-service) se habilitó la integración con métricas personalizadas obtenidas desde Prometheus. Esto permite escalar no solo por CPU, sino también según eventos específicos de negocio, como el número de pedidos pendientes o transacciones simultáneas.

Los valores de CPU target se ajustaron de manera conservadora, considerando los recursos limitados del cluster de desarrollo, evitando así saturar los nodos durante las pruebas.

Pruebas de escalamiento

Para validar el comportamiento de HPA se utilizó la herramienta hey, simulando tráfico HTTP hacia los endpoints de los microservicios. El objetivo de las pruebas fue verificar que los HPA reaccionaran correctamente ante aumentos de carga y que las réplicas se ajustaran dinámicamente.

Debido a la limitación de recursos del cluster local, no se realizaron pruebas de carga exhaustivas. El propósito principal fue demostrar la funcionalidad de escalado automático, no alcanzar el límite máximo de rendimiento del cluster.

Además, para pruebas de estrés más potentes se documentó el uso de **k6**. Se cuenta con una carpeta k6-test que incluye dos archivos de prueba: primero verifica la conectividad con los

microservicios y luego ejecuta la prueba de carga. Esto permite simular escenarios de tráfico más realistas y evaluar el comportamiento del cluster bajo carga elevada.

Observaciones

La integración de HPA con métricas personalizadas permite que el cluster escale según métricas de negocio relevantes, no únicamente por CPU o memoria.

La configuración utilizada asegura que los microservicios puedan adaptarse a cambios temporales en la carga sin necesidad de intervención manual.

El uso de herramientas de simulación de carga, aunque limitado, permitió validar que los HPA respondieran correctamente a picos de tráfico y que las métricas de Prometheus fueran capturadas de forma efectiva.

Limitaciones:

La principal limitación durante el desarrollo del proyecto estuvo relacionada con los recursos físicos del hardware disponible. Esto impactó en varios aspectos:

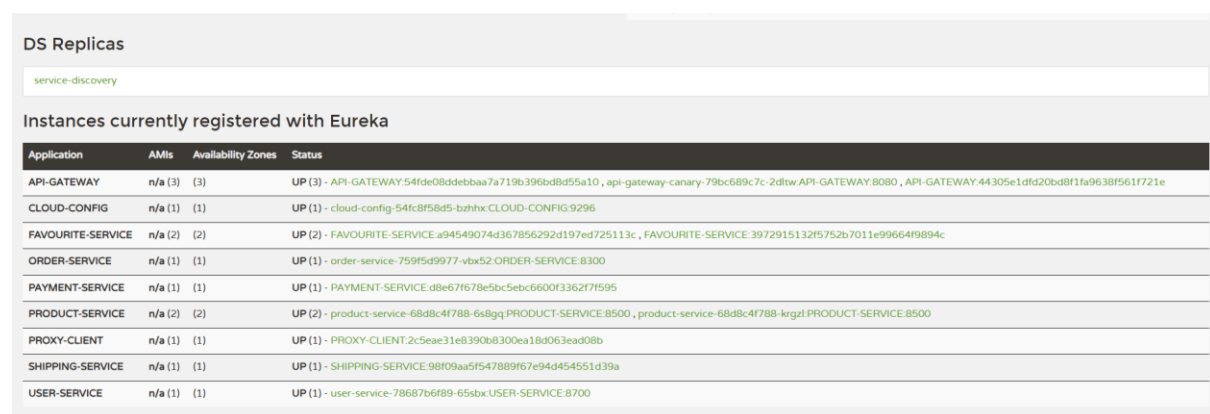
- Algunos microservicios y configuraciones se ejecutaron en menor escala de la que se tendría en un entorno de producción.
- Las pruebas de carga y estrés, aunque funcionales, fueron limitadas en alcance para no saturar el cluster de desarrollo.
- Ciertos procesos o experimentos se documentaron en lugar de implementarse completamente, con el objetivo de preservar los recursos y mantener la estabilidad del cluster.

A pesar de estas limitaciones, el proyecto logró validar los objetivos principales, incluyendo la configuración de HPA, integración con métricas de Prometheus y la capacidad de simular tráfico con hey y k6, asegurando la operatividad y escalabilidad básica de los microservicios.

El proyecto utiliza Eureka como servicio de descubrimiento para todos los microservicios. Esto permite que cada servicio se registre automáticamente y sea localizado por otros componentes del cluster, como el API Gateway y los microservicios dependientes.

En la ilustración presentada a continuación, se evidencia que algunos microservicios cuentan con más de una réplica registrada en Eureka. Esto es consecuencia directa de la configuración de Horizontal Pod Autoscaler (HPA) y del nivel actual de tráfico hacia dichos servicios.

De modo que, gracias a Eureka, las solicitudes pueden ser distribuidas entre múltiples instancias, asegurando alta disponibilidad y tolerancia a fallos.



DS Replicas			
service-discovery			
Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (3)	(3)	UP (3) - API-GATEWAY:54fde08ddebbaa7a719b396bd8d55a10 , api-gateway-canary-79bc689c7c-2dtw API-GATEWAY:8080 , API-GATEWAY:44305e1dfd20bd8f1fa9638f561f721e
CLOUD-CONFIG	n/a (1)	(1)	UP (1) - cloud-config-54fc8f58d5-bzhhx CLOUD-CONFIG:9296
FAVOURITE-SERVICE	n/a (2)	(2)	UP (2) - FAVOURITE-SERVICE:a94549074d367856292d197ed725113c , FAVOURITE-SERVICE:3972915132f5752b7011e99664f9894c
ORDER-SERVICE	n/a (1)	(1)	UP (1) - order-service-759f5d9977-vbx52 ORDER-SERVICE:8300
PAYMENT-SERVICE	n/a (1)	(1)	UP (1) - PAYMENT-SERVICE:d8e67f678e5bc5ebc6600f3362f7f595
PRODUCT-SERVICE	n/a (2)	(2)	UP (2) - product-service-68d8c4f788-6s8gq PRODUCT-SERVICE:8500 , product-service-68d8c4f788-kgz1 PRODUCT-SERVICE:8500
PROXY-CLIENT	n/a (1)	(1)	UP (1) - PROXY-CLIENT:2c5eae31e8390b8300ea18d063ead08b
SHIPPING-SERVICE	n/a (1)	(1)	UP (1) - SHIPPING-SERVICE:98f09aa5f547889f67e94d454551d39a
USER-SERVICE	n/a (1)	(1)	UP (1) - user-service-78687b6f89-65sbc USER-SERVICE:8700

Ilustración 15. Servicios registrados en Eureka.

Validación en dashboards y alertas

Para confirmar que los servicios y métricas se estaban registrando correctamente, se ejecutaron scripts manuales con consultas sobre los microservicios.

Esto permitió activar las alertas designadas en Prometheus, asegurando que los dashboards mostraran datos reales y que el sistema de monitoreo estuviera operativo.

De esta manera, se validó la integración entre Eureka, HPA, Prometheus y los dashboards de Grafana, confirmando que tanto la información de réplica como las métricas personalizadas se registran y visualizan correctamente.

```

Intento 10 -> Código HTTP: 503

=====
Consultando 10 veces: https://ecommerce.local/payment-service/api/payments
=====
Intento 1 -> Código HTTP: 200
Intento 2 -> Código HTTP: 200
Intento 3 -> Código HTTP: 200
Intento 4 -> Código HTTP: 200
Intento 5 -> Código HTTP: 200
Intento 6 -> Código HTTP: 200
Intento 7 -> Código HTTP: 200
Intento 8 -> Código HTTP: 200
Intento 9 -> Código HTTP: 200
Intento 10 -> Código HTTP: 200

=====
Consultando 10 veces: https://ecommerce.local/order-service/api/orders
=====
Intento 1 -> Código HTTP: 200
Intento 2 -> Código HTTP: 200
Intento 3 -> Código HTTP: 200
Intento 4 -> Código HTTP: 200
Intento 5 -> Código HTTP: 200
Intento 6 -> Código HTTP: 200
Intento 7 -> Código HTTP: 200
Intento 8 -> Código HTTP: 200

```

Ilustración 16. Generación de tráfico a los diferentes endpoints.

Para facilitar el monitoreo de flujos y depuración de solicitudes entre microservicios, el proyecto integra Zipkin como sistema de trazabilidad distribuida.

Cada microservicio envía información de trazas a Zipkin sobre las solicitudes que procesa. De modo que es posible observar el recorrido de cada request a través de múltiples servicios, identificando latencias, cuellos de botella y posibles errores. Y así, zipkin se integra con los dashboards de Grafana para complementar las métricas de Prometheus, proporcionando una visión completa del rendimiento y comportamiento del sistema.

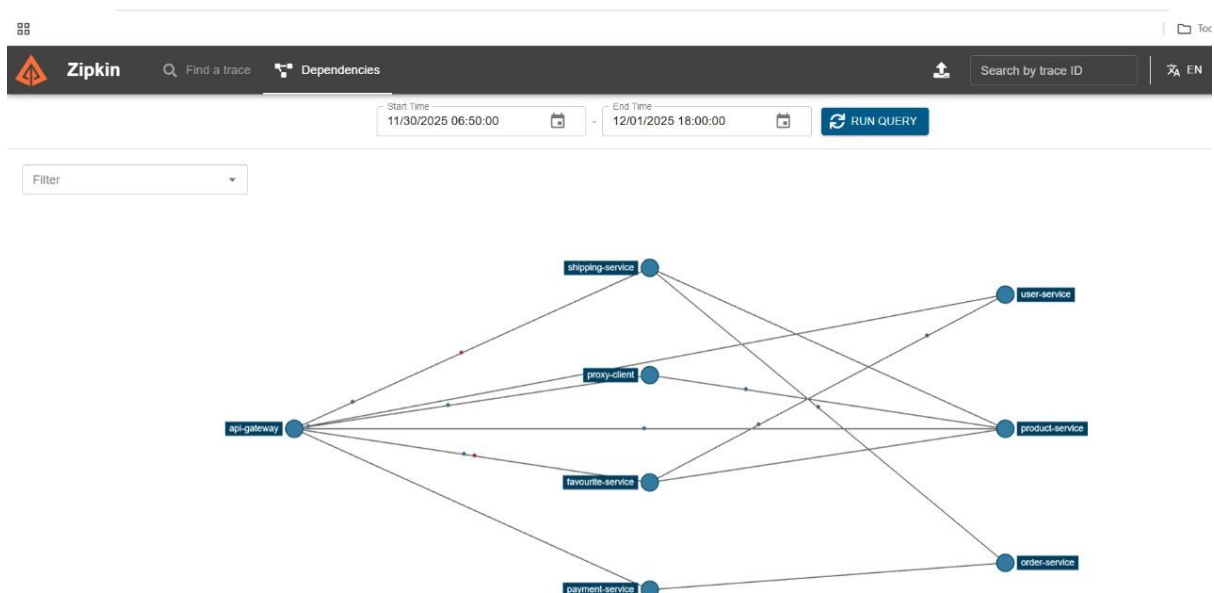


Ilustración 17. Visualización de trazas en zipkin.

Guia rápida de instalación:

Preparación del entorno

Antes de levantar el proyecto, asegúrase de tener:

- Minikube instalado y activo.
- Helm instalado y configurado.
- kubectl apuntando al cluster correcto.
- Acceso a los manifiestos en la carpeta infrastructure y a los Helm charts (helm-charts/core y helm-charts/services).

Si el cluster fue eliminado o quieres empezar limpio:

```
minikube delete -p ecommerce-app
```

```
minikube start -p ecommerce-app --driver=docker --cpus=4 --memory=8g
```

Se recomienda asignar suficiente CPU y memoria para que los microservicios y Prometheus puedan ejecutarse sin saturar el cluster.

Una vez creado, valide la dirección ip mediante la ejecución del comando:

```
minikube ip -p ecommerce-app
```

Posterior a ello, configure /etc/hosts para apuntar ecommerce.local a esta IP:

```
<minikube-ip> ecommerce.local
```

Desplegar infraestructura base

Desde la carpeta infrastructure (excepto prometheus) aplica los recursos:

```
kubectl apply -f <archivo>.yaml -n dev
```

Recomendación: Siempre especificar el namespace (- <namespace>) para evitar conflictos con otros entornos.

Esto incluye recursos como ConfigMaps, Secrets, Ingress (API Gateway), y cualquier recurso compartido por los microservicios.

Instalar Helm Charts del core

Dirigirse a la carpeta helm-charts/core.

Instalar los charts del núcleo, por ejemplo:

```
helm install cloud-config ./cloud-config -n dev  
helm install service-discovery ./service-discovery -n dev  
helm install zipkin ./zipkin -n dev
```

Esto levantará el core del sistema, incluyendo la configuración central y descubrimiento de servicios.

Instalar Helm Charts de servicios

Cambiar a la carpeta helm-charts/services. E instale los charts, por ejemplo:

```
helm install product-service ./product-service -n dev  
helm install order-service ./order-service -n dev  
helm install payment-service ./payment-service -n dev
```

Puedes modificar el namespace si se quiere un entorno diferente al dev.

Los servicios se registrarán automáticamente en Eureka y estarán listos para ser consumidos por el API Gateway.

Activar Prometheus y Grafana

Para habilitar monitoreo y dashboards:

```
helm install prometheus prometheus-community/prometheus -n dev
```

```
helm install grafana grafana/grafana -n dev
```

Para aplicar valores personalizados

```
helm upgrade prometheus prometheus-community/prometheus -f infrastructure/prometheus/metrics-values.yaml -n monitoring
```

```
helm upgrade grafana grafana/grafana -f infrastructure/prometheus/grafana-values.yaml -n monitoring
```

Esto permitirá:

- Visualizar métricas de CPU, memoria y métricas personalizadas.
- Activar alertas de Prometheus.

Validar el sistema

Verificar que los pods estén corriendo:

```
kubectl get pods -n dev
```

Probar conectividad interna (desde dentro del cluster):

```
kubectl run curlpod --rm -it --image=curlimages/curl -n dev -- /bin/sh  
curl http://api-gateway:8080/product-service/api/products
```

Revisar servicios registrados en Eureka:

```
kubectl port-forward svc/service-discovery 8761:8761 -n dev
```

Accede a `http://localhost:8761` para ver todos los microservicios y sus réplicas.

Probar autoescalamiento y carga:

Usar hey para generar tráfico simulado y verificar HPA:

```
hey -z 30s -c 5 http://api-gateway:8080/product-service/api/products
```

Usar k6 para pruebas de estrés más avanzadas:

```
cd k6-test
```

```
k6 run test-connectivity.js
```

Primero se valida la conectividad, luego se realiza la prueba de carga.

Ver trazabilidad con Zipkin:

Zipkin está desplegado como servicio ClusterIP en el namespace dev.

Para ver trazas:

```
kubectrl port-forward svc/zipkin 9411:9411 -n dev
```

Accede a `http://localhost:9411` y podrá ver el recorrido de las solicitudes entre los microservicios.

Cloud config server

Cloud Config Sever es un repositorio privado, por lo que para que el proyecto funcione correctamente es necesario crear los Secrets correspondientes con credenciales y datos sensibles.

Dado que repositorio al que apunta Cloud Config actualmente es privado, por razones de seguridad, no se proporciona el token de acceso.

Antes de desplegar los microservicios que dependen de Cloud Config (por ejemplo, api-gateway), asegúrese de crear los Secrets en el namespace correspondiente :

```
kubectrl create secret generic cloud-config-credentials \  
  --from-literal=username=<usuario> \  
  --from-literal=password=<contraseña> \  
  -n dev
```

Esto garantiza que los microservicios puedan obtener sus configuraciones de manera segura y sin exponer información sensible en el repositorio.

Además, asegúrese de contar con el resto de las credenciales necesarias, en caso de no tenerlas puede presentar problemas asociados con los microservicios.

Conclusiones

La implementación de este proyecto permitió consolidar una plataforma integral basada en microservicios, siguiendo los principios de arquitectura cloud-native y empleando Kubernetes como herramienta central para la gestión, despliegue y operación del sistema. A través del diseño estructurado del clúster, la separación de entornos, la centralización de configuraciones y la incorporación de medidas avanzadas de seguridad, se logró construir una infraestructura sólida, modular y alineada con buenas prácticas de la industria.

La adopción de ConfigMaps, Secrets, rotación automática de credenciales, Ingress con TLS y políticas de seguridad demostró la importancia de gestionar correctamente los datos sensibles y de limitar los permisos según el principio de privilegio mínimo. Asimismo, el uso de herramientas como Helm, Prometheus, Grafana, Zipkin y Trivy permitió fortalecer los procesos de despliegue, observabilidad y auditoría del sistema, garantizando que cada microservicio pudiera operar de manera consistente, monitoreada y segura.

De igual forma, la integración de un pipeline CI/CD con estrategias de despliegue progresivo, validaciones automatizadas y rollback asistido mostró la relevancia de automatizar las operaciones para reducir errores humanos y mejorar la confiabilidad del sistema. La incorporación de mecanismos de autoscaling basados tanto en métricas estándar como en métricas personalizadas, junto con pruebas de rendimiento, permitió evaluar la capacidad del sistema para adaptarse a diferentes cargas, manteniendo niveles adecuados de disponibilidad y eficiencia.

En conjunto, este proyecto no solo permitió desplegar exitosamente un ecosistema funcional de microservicios, sino que también evidenció la importancia de combinar prácticas de infraestructura, seguridad, automatización y observabilidad en una solución unificada. El resultado es una plataforma robusta, escalable y preparada para evolucionar hacia entornos más complejos o productivos, ofreciendo una base sólida para futuros desarrollos y mejoras dentro del ámbito de las arquitecturas distribuidas.